DocAssist AI

Clinic Assistant AI Receptionist

Architecture, Technology Stack & Voice Platform Comparison

Prepared for the week 3 / day 2 proof of concept • 27 October 2026

Purpose: explain how the system works, the advantages and limitations of its technology choices, and why PostgreSQL and Vapi are suitable for this project.

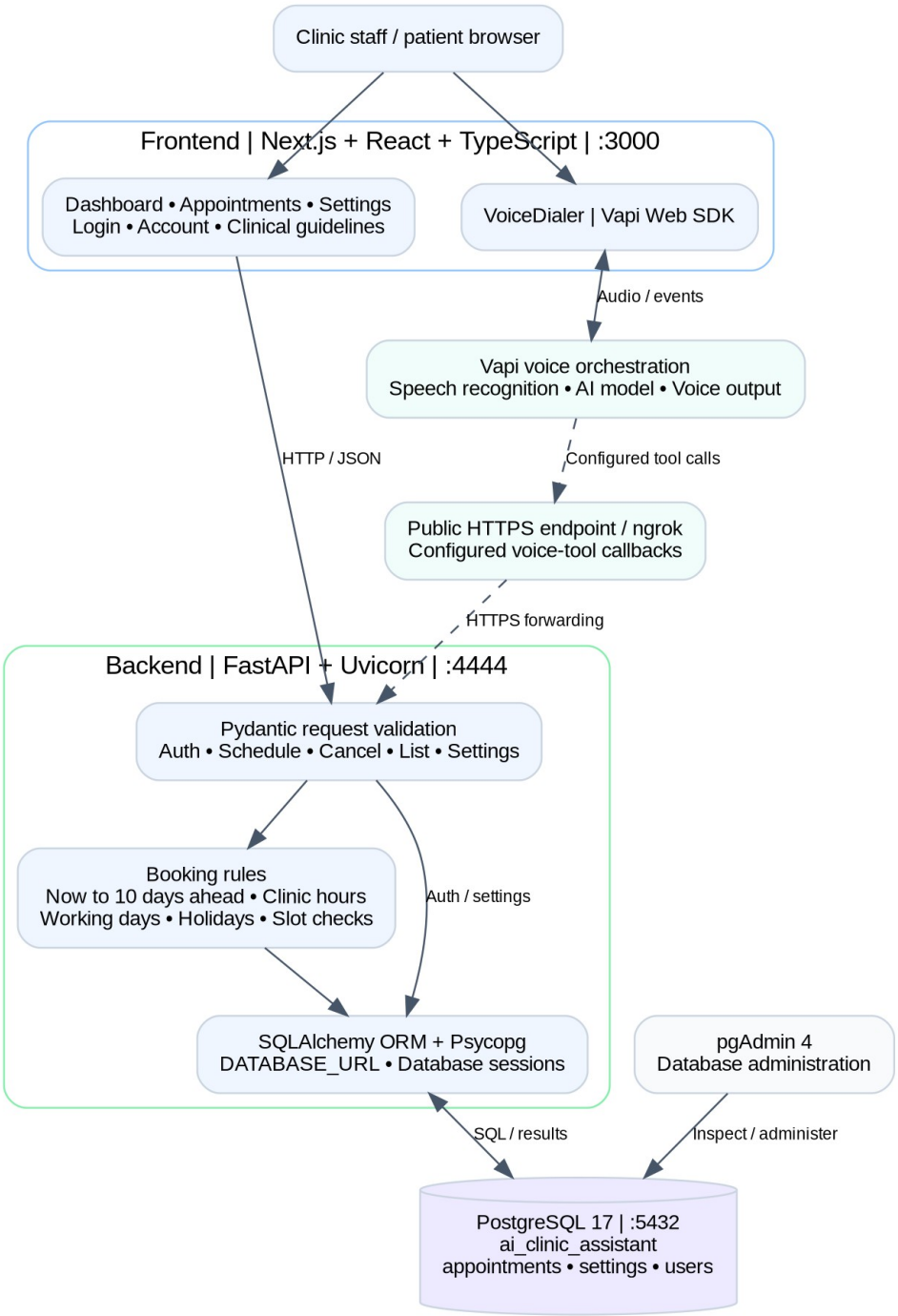
1. Project scope and evidence

The application combines a clinic dashboard with a conversational voice receptionist. Patients can request bookings and cancellations; staff can inspect appointments and configure clinic hours. FastAPI applies booking rules and stores records in PostgreSQL.

This document combines the inspected test2-branch implementation with the PostgreSQL migration and rolling 10-day validation completed locally by the project owner. The cloud checkout is not proof of those local changes. External Vapi assistant configuration and production deployment were not independently verified.

The active backend is FastAPI. The legacy_nestjs_backend directory is an older implementation and is not part of the active design. No clinical diagnosis capability is claimed; clinical guidelines are frontend reference content.

2. Architecture design



Solid arrows show the application connections. Dashed arrows show the voice-tool path that must be configured in Vapi. ngrok is a development tunnel; a production backend should have a stable HTTPS address. Responses return through the same request path.

Layer responsibilities

Layer	Responsibility
Frontend	Next.js and React render dashboard pages, charts, forms and the browser voice interface. The browser calls FastAPI through

Layer	Responsibility
	config.ts.
Voice platform	Vapi coordinates audio, AI conversation and tool calls. It requests backend actions but does not decide whether a booking is valid.
Backend	FastAPI validates requests, verifies login credentials, enforces business rules and coordinates database operations.
Persistence	SQLAlchemy models and sessions access PostgreSQL through the Psycopg driver. PostgreSQL stores the durable records.
Administration	pgAdmin connects directly to PostgreSQL for inspection and administration; application requests do not pass through pgAdmin.

3. Technology stack: advantages and disadvantages

Technology	Role and advantages	Disadvantages / tradeoffs
Next.js + React	Frontend routing and reusable interactive components; broad ecosystem; convenient dashboard and voice SDK integration.	Client/server boundaries and hydration need care; development Fast Refresh can complicate audio SDK lifecycle.
TypeScript	Checks frontend data shapes and catches many integration errors during development.	Types add maintenance work and do not validate external JSON at runtime by themselves.
Tailwind CSS	Utility classes support fast, consistent interface styling and responsive layouts.	Long class lists can reduce readability; layout still needs explicit sizing for charts.
Recharts	React chart components simplify dashboard summaries and visualizations.	Responsive charts depend on measurable container dimensions; empty/loading states need handling.
FastAPI + Uvicorn	Typed Python API development, automatic OpenAPI documentation and convenient request handling.	Deployment, access control and blocking database operations require deliberate design; one process is not a complete production architecture.
Pydantic	Validates incoming data types and provides clear request errors.	Business rules such as booking windows and slot conflicts still need backend logic.
SQLAlchemy	Centralizes models, sessions and queries; helped reduce SQLite-to-PostgreSQL migration changes.	ORM usage does not remove database differences; transactions, indexes and migrations still need attention.
Psycopg	PostgreSQL driver used by SQLAlchemy to communicate with the database.	Adds a dependency and connection configuration; driver choice must match the connection URL.
PostgreSQL 17	Transactional, relational storage with constraints and strong support for concurrent users.	Requires a running service, credentials, backups and operational maintenance.
pgAdmin 4	GUI for viewing records, running SQL and administering PostgreSQL.	Separate management tool; it does not host the application or replace the PostgreSQL server.
Vapi Web SDK	Browser voice sessions, transcripts and call events integrate with the React interface.	External-service dependency; call errors and SDK lifecycle need monitoring.

Technology	Role and advantages	Disadvantages / tradeoffs
ngrok — optional	Makes the local backend reachable by external voice tools over HTTPS during development.	Tunnel availability and URL changes can break callbacks; production needs stable hosting.

4. Why PostgreSQL was chosen

SQLite was useful for the initial prototype because it needed no separate database server. PostgreSQL is a better fit as the clinic system grows to multiple staff sessions, simultaneous booking requests and separately deployed services.

- Concurrent access: PostgreSQL supports multiple clients and simultaneous transactions more naturally than a single local SQLite file.
- Data integrity: primary keys, unique constraints, foreign keys and transactions help keep stored records consistent when explicitly configured.
- Deployment: the database can run separately from the API and be accessed by multiple API instances.
- Operations: PostgreSQL supports established backup, restore, indexing and monitoring workflows.
- Migration fit: the project already uses SQLAlchemy, so its existing model structure can be retained while changing the engine URL and driver.

Important limitation: changing databases alone does not prevent double bookings. A check-then-insert sequence can race under concurrent requests. A database constraint or suitable transactional locking is needed for a strict guarantee. The current exact-start-time conflict check also does not prove that overlapping appointment durations are prevented.

SQLite	PostgreSQL
Embedded database stored in a local file.	Separate server accessed through authenticated connections.
Minimal setup; good for demos and small local workloads.	More operational setup; better fit for shared application workloads.
Writes are serialized; contention becomes a concern as concurrent writes increase.	Supports concurrent transactional workloads with more control.
File persistence must be managed with the application host.	Can be hosted and backed up independently of the API.

Migration details

Database: ai_clinic_assistant. Local server: localhost:5432. SQLAlchemy driver prefix: postgresql+psycopg. Supply DATABASE_URL through environment configuration; do not commit credentials. SQLite-specific connect_args such as check_same_thread must be removed.

Creating PostgreSQL tables does not transfer SQLite records. Existing users, appointments and settings require a separate data migration with row-count and field checks. Base.metadata.create_all creates missing tables; it is not a replacement for versioned schema migrations such as Alembic.

5. Voice service comparison

These are architectural comparisons, not claims that one provider always has better latency, quality or price. Available integrations, plans, trial credits and regional coverage change. Validate each provider against the same real booking scenarios.

Vapi

Managed voice-agent orchestration with browser SDK integration and configurable backend tools.

Advantages:

- Fast integration with the current React VoiceDialer.
- Coordinates speech input, AI conversation and spoken output.
- Supports backend tool calls for booking workflows and call diagnostics.
- Supported model and voice choices provide configuration flexibility.

Disadvantages:

- Ongoing usage costs and third-party availability dependency.
- Provider configuration, tool schemas and SDK lifecycle still require engineering.
- Multiple connected providers can make cost and failure diagnosis more complex.

Project fit: Strong fit for this POC because the SDK and backend integration already exist.

Retell AI

Managed conversational voice-agent platform; verify its current browser, telephony and tool integration options for your plan.

Advantages:

- Reduces the need to assemble the voice pipeline yourself.
- Provides a managed approach to voice-agent deployment and call inspection.
- Reasonable alternative to evaluate using the existing FastAPI business logic.

Disadvantages:

- Requires replacing the current SDK integration and adapting tools/events.
- Usage pricing, limits and supported integrations depend on the current plan.
- External platform dependence remains after migration.

Project fit: Useful comparison candidate when evaluating another managed voice platform.

Bland AI

Managed voice automation platform with a strong phone-call workflow focus.

Advantages:

- Can simplify phone-based inbound or outbound automation.
- Managed calling workflows reduce low-level telephony work.
- Backend integration can preserve the booking rules in FastAPI.

Disadvantages:

- May require more adaptation for the current browser-first interaction.
- Calling, geographic coverage, features and limits must be verified.
- Provider-specific workflows introduce migration and maintenance work.

Project fit: Evaluate when phone-call automation is the primary requirement.

OpenAI Realtime API

API for low-latency audio interaction; the application builds more of the session and tool-execution layer.

Advantages:

- More direct control of the voice session, instructions and tool handling.
- Supports real-time audio interaction without assembling every speech stage separately.

- Can connect the same booking backend through application-managed tool execution.

Disadvantages:

- Requires more work for session lifecycle, interruption handling, reconnection and monitoring.
- Phone access requires a supported telephony/SIP integration.
- Browser authentication must use supported short-lived credentials; never expose a long-lived secret key.

Project fit: Suitable when custom interaction control justifies additional implementation work.

Twilio + AI services

A composable design using Twilio for telephony and chosen AI/speech services for the conversation.

Advantages:

- Greater control over phone numbers, call routing and provider selection.
- Useful for integrating existing phone workflows.
- Can preserve FastAPI and PostgreSQL while customizing the voice stack.

Disadvantages:

- Usually the most integration work among these options.
- The application must coordinate audio streaming, interruptions, tool calls and failure recovery.
- Costs and diagnostics span telephony, models, speech services and hosting.

Project fit: Suitable when telephony customization and control outweigh implementation simplicity.

6. Why Vapi was chosen for this project

Vapi is a practical choice because the POC needs a working voice receptionist quickly, and the project already integrates @vapi-ai/web in VoiceDialer.tsx. It connects a browser voice interface with backend booking tools while allowing the database and business rules to remain under application control.

- Implementation speed: reuse the current voice interface rather than building the whole audio pipeline.
- Separation of responsibilities: Vapi manages conversation; FastAPI validates and executes requests; PostgreSQL persists the result.
- Debugging: inspect call events, tool responses and call-ended reasons when a booking fails.
- Replaceability: preserving backend endpoints makes a future voice-platform migration possible, although tool schemas and SDK code will need adaptation.

Selection statement: “I chose Vapi to reduce voice integration work and connect a conversational receptionist to my React frontend and FastAPI backend. I chose PostgreSQL for durable relational storage and concurrent application access. The backend remains the authority for booking rules.”

Tradeoff: this is a project-fit decision, not proof that Vapi is universally superior. Compare successful booking rate, response latency, interruption handling, total cost, privacy requirements and ease of debugging before a production commitment.

7. Appointment lifecycle and validation

- The patient speaks through the browser voice interface; Vapi collects name, reason and requested date/time.
- A configured Vapi tool calls the public scheduling endpoint, forwarded through ngrok during local development.
- Pydantic validates the request. The backend normalizes the timestamp to clinic time.
- The backend accepts only now ≤ appointment time ≤ now + 10 days, then checks clinic settings, holidays, working days, opening hours and existing bookings.

- SQLAlchemy inserts the accepted appointment into PostgreSQL and commits the transaction.
- The backend returns confirmation or a validation error. Vapi explains the outcome to the patient.
- The dashboard requests appointments over HTTP and displays the returned records. Automatic push refresh is not established by the inspected implementation.

Timezone assumption: the supplied booking update uses Pakistan time (UTC+05:00), interpreting timestamps without an offset as clinic-local time. The current model uses timestamp without time zone. A future production design should explicitly document the clinic timezone and use a consistent storage and conversion policy across every endpoint.

8. Database design and API map

Table	Fields	Purpose
appointments	id, patient_name, reason, start_time, canceled, created_at	Booking records and cancellation state.
settings	id, start_hour, end_hour, slot_duration_minutes, working_days, holidays	Clinic schedule configuration.
users	id, name, email, password_hash, created_at	Staff accounts and stored password hashes.

The inspected models do not define foreign-key relationships among these tables. Appointments store patient names rather than linking to a separate patient table. A single settings row is assumed by scalar_one_or_none(), but singleton enforcement needs deliberate design.

Endpoint	Method	Responsibility
/auth/signup	POST	Create a staff account.
/auth/login	POST	Verify credentials and return user information.
/schedule_appointment/	POST	Validate and save an appointment.
/cancel_appointment/	POST	Mark matching appointments canceled.
/list_appointments/	GET / POST	Return appointments, optionally filtered by date.
/check_doctor_availability/; /doctor_availability	GET / POST	Aliases sharing the listing handler; inspect semantics before describing these as free-slot computation.
/settings/	GET / POST	Read or update clinic configuration.
/ws	WebSocket	Connection manager infrastructure; booking broadcasts and page subscriptions were not established.

9. Current limitations and production next steps

- Authentication: the inspected frontend stores user_session in localStorage. Login success and a frontend guard do not by themselves secure API routes. Add server-verified sessions/tokens and authorization to protected operations.
- Booking integrity: enforce concurrency-safe slot reservations in the database, account for slot duration and ensure appointments finish before clinic closing time.
- Security and privacy: protect credentials and voice-tool endpoints, restrict access to patient information, define retention rules and review vendor data-processing terms.

- Schema evolution: use versioned migrations, backups and restore tests. Do not assume create_all updates existing table definitions.
- Voice reliability: guard against duplicate call starts, manage SDK lifecycle and use the actual call-ended reason to diagnose failures.
- Frontend: size chart containers explicitly, handle empty/error states and avoid treating development duplicate fetch logs as database failures.
- Deployment: replace development-only processes and tunnels with stable HTTPS hosting, managed configuration and monitoring. The in-memory WebSocket manager alone does not coordinate multiple API instances.

10. Pricing, trials and reference links

No current prices, free-credit amounts or credit-card requirements are asserted here. Live provider pages could not be verified in this session. In particular, the previously discussed Retell $10 credit offer is historical/unverified and must not be treated as a guaranteed current offer. Check signup terms, expiry, per-minute components, phone-number costs and overage rules before testing.

Provider / technology	Official reference
Vapi	https://docs.vapi.ai/ | https://vapi.ai/pricing
Retell AI	https://docs.retellai.com/ | https://www.retellai.com/pricing
Bland AI	https://docs.bland.ai/ | https://www.bland.ai/
OpenAI Realtime	https://platform.openai.com/docs/guides/realtime | https://openai.com/api/pricing/
Twilio	https://www.twilio.com/docs/voice | https://www.twilio.com/en-us/voice/pricing
PostgreSQL / SQLAlchemy	https://www.postgresql.org/docs/ | https://docs.sqlalchemy.org/
FastAPI / Next.js	https://fastapi.tiangolo.com/ | https://nextjs.org/docs

Evidence basis: selected repository frontend package/configuration and components, FastAPI routes, SQLAlchemy model definitions, the user-provided backend, the generated 10-day booking update and the user's PostgreSQL screenshots. Provider links are reference destinations, not evidence of a live pricing check.